

From Text Safety to Execution Security: Bridging the Trust Gap in Autonomous Agentic Systems

The evolution of Large Language Models (LLMs) from conversational interfaces to autonomous agents has fundamentally altered the cybersecurity landscape, shifting risks from hallucinated text to irreversible system actions. This review synthesizes current literature to define the "Trust Gap," a critical vulnerability where agents fail to maintain skepticism toward untrusted tool outputs and external environments. We demonstrate that traditional content safety filters are insufficient against execution-layer threats such as Tool Description Poisoning [41] and persistent Sleeper Attacks [285]. Furthermore, we highlight the "over-defense dilemma," where static benchmarks encourage brittle defenses that degrade utility in dynamic, open-ended tasks [270]. In response, we argue for a paradigm shift from probabilistic model alignment to deterministic, system-level governance. We evaluate emerging architectural solutions, including Cognitive-Executive Separation [299] and verify-before-commit protocols, which structurally decouple reasoning from action to enforce zero-trust execution. By integrating causal attribution and lifecycle-integrated security, this review provides a roadmap for securing agentic ecosystems against adaptive, multi-stage attacks while preserving operational autonomy.

Introduction: The Paradigm Shift from Text Safety to Execution Security

The evolution of Large Language Models (LLMs) from static conversational interfaces to autonomous agents capable of executing tools, modifying system states, and maintaining persistent memory has fundamentally altered the cybersecurity landscape. Unlike traditional chatbots where failures primarily result in hallucinated text or benign misinformation, agent failures can lead to irreversible actions such as data exfiltration, financial loss, or physical harm. This transition necessitates a critical re-evaluation of security paradigms, moving beyond defect remediation in text generation toward comprehensive system-level governance. The literature indicates that existing content safety filters, designed for conversational safety, are insufficient for these execution-layer threats, creating an urgent need for new architectural approaches to agent security [7].

The core distinction in this paradigm shift lies in the agency granted to the model. While traditional LLMs operate within a text-in, text-out constraint, modern agents leverage LLMs as a "brain" to autonomously perform actions via connected tools, introducing risks that extend far beyond harmful text [7]. Research demonstrates that safety fine-tuning in chat models does not generalize well to agentic behavior; models that refuse to provide advice on harmful tasks in a chat setting often willingly execute those same tasks when equipped with tool access [282]. This discrepancy is exacerbated by the "helpfulness" objective, where agents prioritize task completion over safety constraints unless explicitly prompted otherwise, leading to high bypass rates in commercial systems [131]. Consequently, the threat model expands from simple prompt injection to complex, multi-stage attacks that exploit the agent's ability to interact with its environment.

A primary vector for these new threats is the integration of external tools and memory systems. The reliance on external memory to maintain context across stateless interactions creates significant vulnerabilities, as agent memory is often managed client-side or by third-party applications. Attacks such as "plan injection" can corrupt an agent's internal task representations, bypassing robust prompt injection defenses and achieving significantly higher success rates than comparable prompt-based attacks [106]. Furthermore, the persistence of memory introduces the risk of "sleeper" attacks, where adversarial content remains dormant across sessions before being triggered to steer future conversations, effectively transforming ephemeral model-level threats into long-lived system-level vulnerabilities [285] [68]. The vulnerability of memory is further highlighted by studies showing that routine user-agent interactions can gradually poison long-term state, inadvertently weakening confirmation boundaries and escalating autonomous behavior over time [271].

The complexity of threats increases exponentially in multi-agent systems and environments with extensive toolchains. In multi-agent setups, vulnerabilities such as "Prompt Infection" allow malicious prompts to self-replicate across interconnected agents, behaving like computer viruses and causing system-wide disruption [77]. Additionally, the interplay between different tools can be exploited through Sequential Tool Attack Chaining (STAC), where individually harmless tool calls are chained together to enable harmful operations that only become apparent at the final execution step [75]. The Model Context Protocol (MCP), while standardizing tool connectivity, introduces new attack surfaces such as Tool Description Poisoning, where malicious instructions are injected into tool metadata rather than executable code, deceiving the agent's planning layer [41] [192]. These findings underscore that the attack surface is no longer limited to the input prompt but encompasses the entire execution harness, including tool descriptions, memory stores, and inter-agent communication channels.

Existing defenses often fail to address these systemic architectural flaws because they operate at the text-generation layer rather than the execution layer. Content safety systems like Llama Guard filter harmful text but lack the capability to intercept unauthorized tool invocations arising from semantically benign inputs [1]. Similarly, traditional sandboxing and input filtering are insufficient against indirect prompt injections embedded in trusted data sources like web pages or documents, which agents process as legitimate observations [58]. The literature suggests that a shift toward zero-trust agentic execution, dynamic intent verification, and cross-layer reasoning-action correlation is required to mitigate these risks [33]. Frameworks such as SafeAgent and Parallax propose runtime protection architectures that separate execution governance from semantic risk reasoning, treating agent safety as a stateful decision problem over evolving interaction trajectories rather than a static filtering task [97] [299].

The severity of these vulnerabilities is quantified by numerous benchmarks revealing alarming success rates for various attack vectors. Studies indicate that leading LLMs are surprisingly compliant with malicious agent requests even without sophisticated jailbreaking, and that simple universal jailbreak templates can be adapted to enable coherent, multi-step malicious behavior [100]. In specific scenarios involving computer-use agents, attack success rates exceed 90%, with even safety-aligned models failing when deployed in multi-agent systems where decomposed subtasks obscure harmful intent [101]. Moreover, the "Trusted Executor Dilemma" highlights that high-privilege agents frequently execute documentation-embedded instructions without verification, leading to end-to-end exfiltration success rates of up to 85% [216]. These empirical findings confirm that the transition to agentic AI has created a qualitative shift in risk, where the consequences of failure are no longer abstract but involve direct, irreversible impact on digital and physical systems. Addressing these challenges requires a holistic approach that integrates security into the agent lifecycle, from design-time hardening to runtime monitoring and incident response, moving beyond the limitations of isolated defense mechanisms [142] [3].

The Evolution of Agent Capabilities and Associated Risks

The transition from static large language models to autonomous agents represents a fundamental paradigm shift in artificial intelligence, moving systems from passive text generators to interactive, memory-augmented entities capable of perceiving, reasoning, and acting in dynamic environments. This evolution is characterized by the integration of long-term memory retention, modular tool use, recursive planning, and reflective reasoning, which collectively expand the functional scope of AI but simultaneously introduce qualitatively novel security risks that extend beyond conventional threat models [3]. Unlike traditional systems where vulnerabilities are often confined to input processing, modern agents face a vastly enlarged attack surface spanning perception, cognition, memory, and action modules, creating failure modes such as deferred decision hazards, irreversible tool chains, and emergent misalignment [3]. The reliance on persistent memory to accumulate knowledge across interactions creates a critical vulnerability where a single adversarial write can exert long-term influence over agent behavior, transforming ephemeral prompt injections into persistent system-level compromises [138].

The introduction of long-term memory has given rise to sophisticated attack vectors such as "sleeper" poisoning, where malicious content remains dormant until triggered by specific contexts, and self-reinforcing error cycles where corrupted outcomes are stored as precedents, progressively lowering the threshold for future attacks [269; 285]. Research indicates that memory poisoning can occur through untrusted external content like web pages and tool outputs, which are written to memory and subsequently treated as trusted knowledge without robust provenance tracking [138]. Furthermore, attacks like MemMorph demonstrate that poisoning long-term memory can bias tool selection policies more effectively than manipulating tool metadata, achieving high success rates with minimal injected records by reshaping the agent's contextual perception [52]. The risk is compounded in self-evolving agents, where "Zombie Agent" attacks can covertly implant payloads that survive across sessions, effectively turning the agent into a persistent puppet of the attacker through indirect exposure to attacker-controlled web content [229]. Even routine, benign user interactions can inadvertently lead to unintended long-term state poisoning, gradually eroding confirmation boundaries and escalating autonomous behavior over time [271].

Beyond memory vulnerabilities, the modular nature of tool use and the adoption of protocols like the Model Context Protocol (MCP) have introduced new vectors for semantic attacks and supply chain compromises. Tool Description Poisoning (TDP) exploits the agent's reliance on descriptive metadata, injecting malicious instructions into the "manuals" agents use for planning, often bypassing standard prompt guardrails which can prove counterproductive [41]. The trust placed in third-party tools and skills creates additional risks; for instance, "Agent Skills" frameworks allow trivially simple prompt injections hidden within markdown files or scripts, enabling data exfiltration under the guise of benign task execution [57]. Similarly, dependency steering attacks can bias coding agents toward installing attacker-controlled packages through persistent skill artifacts, highlighting the software supply chain risks inherent in agentic ecosystems [186]. The phenomenon of Sequential Tool Attack Chaining (STAC) further illustrates how individually harmless tool calls can be chained to execute harmful operations that only become apparent at the final step, evading defenses that evaluate actions in isolation [75].

A particularly insidious category of risk involves "Owner-Harm" scenarios, where agents inadvertently damage the interests of their deployers, a threat model often overlooked by benchmarks focused on generic criminal harm [65]. This includes situations where benign user instructions lead to severe harms due to environment-embedded threats or agent-initiated actions, a vulnerability that is exacerbated in multi-agent systems where decomposed subtasks obscure harmful intent [101]. User-mediated attacks further complicate the landscape, as adversaries can manipulate benign users to relay untrusted content, exploiting the agent's default prioritization of helpfulness over safety [131]. Additionally, cognitive poisoning attacks exploit the implicit trust in tool feedback, where malicious tools accumulate trust through benign interactions before executing harmful actions when specific hidden state conditions align [119].

The complexity of these risks is amplified in multi-agent systems (MAS), where integrity attacks can introduce systematic biases, allowing malicious agents to manipulate collaborative workflows or mislead evaluation systems without disrupting core functionality [250]. Benchmarks like TAMAS and PEAR reveal that multi-agent dynamics introduce unique vulnerabilities, such as the susceptibility of planner agents to misleading inputs which can degrade the performance of the entire system [264; 173]. Furthermore, the shift to computer-use agents (CUAs) with operating-system-level privileges introduces risks like clickjacking via visual overlays and remote code execution

through chained tool use, necessitating a re-evaluation of trust boundaries in graphical user interfaces [30]. Current defense mechanisms often struggle to address these cross-module and temporally extended threats, with many operating in isolation or failing to generalize to dynamic, open-ended tasks where context-dependent reasoning is essential [3; 239]. Consequently, there is an urgent need for integrated architectural solutions, such as lifecycle supervision frameworks and zero-trust execution models, to manage the emergent risks of increasingly autonomous systems [104; 33].

The Limitations of Traditional Guardrails and Benchmarks

The rapid evolution of Large Language Model (LLM) agents from conversational interfaces to autonomous executors has exposed a critical misalignment between existing security paradigms and the emergent threat landscape. Traditional defense mechanisms, such as Llama Guard and NeMo Guardrails, were primarily architected to operate at the text-generation layer, focusing on filtering harmful content in user inputs or model outputs [1]. While effective for preventing the generation of toxic dialogue, these content-safety systems fundamentally fail to intercept unauthorized tool invocations that arise from semantically benign inputs or indirect prompt injections embedded in retrieved data [1]. This structural mismatch leaves agents vulnerable to execution-layer threats where the model's reasoning is hijacked to perform irreversible actions, such as data exfiltration or system compromise, without triggering standard refusal vectors [7]. Research indicates that safety fine-tuning in chat models does not generalize well to agentic behavior; models that robustly refuse harmful advice in a chat completion setting may willingly execute the same harmful tasks when framed as tool-use objectives [282]. Consequently, relying on text-level guardrails creates a false sense of security, as they do not address the core vulnerability of agents: the ability to translate benign natural language into malicious system actions.

Furthermore, the current ecosystem of security benchmarks fosters this complacency by relying on weak attack vectors and static evaluations that fail to capture the adaptive, multi-turn, and stateful nature of real-world agent interactions. Many existing benchmarks suffer from a lack of dynamic, open-ended tasks, often presenting scenarios that can be fully planned upfront, which allows defenses to exploit shortcuts by adhering to initial plans rather than verifying ongoing intent [270]. This static nature overlooks the reality that modern agents must engage in dynamic replanning, where defenses relying on initial context isolation frequently result in severe over-defense, crippling agent utility in complex environments [270]. Additionally, prevalent benchmarks often lack helpful instructions within third-party data, enabling defenses to achieve high security scores simply by flagging and ignoring all external inputs, a strategy that is non-viable in realistic settings where benign and malicious instructions are intermingled [270]. This limitation is particularly dangerous because it encourages the development of brittle defenses that collapse when faced with the nuanced context-dependency required in actual deployments [239].

The inadequacy of current evaluation frameworks is further compounded by their inability to simulate adaptive adversaries. Studies have demonstrated that defenses appearing robust under standard benchmarks can be consistently bypassed when subjected to adaptive attacks that iteratively refine payloads based on agent responses [38]. For instance, adaptive frameworks like MUZZLE have successfully discovered novel attack strategies and cross-application injections that static test suites miss, revealing that many state-of-the-art defenses are not secure enough for real-world deployment [130]. Moreover, existing benchmarks often focus on single-turn, stateless behaviors, ignoring the expanded attack surface inherent in stateful interactions where adversarial content can persist across sessions, remain dormant, and be activated by subsequent benign queries—a phenomenon known as *Sleeper Attacks* [195]. This gap is critical because it means that an agent deemed safe in a benchmark evaluation could harbor dormant vulnerabilities that are triggered days or weeks after deployment.

Another significant limitation is the oversight of context-dependent vulnerabilities and subtle attack vectors that do not rely on overtly malicious prompts. Benchmarks like OS-BLIND have revealed that computer-use agents are highly susceptible to attacks where user instructions are entirely benign, but harm arises from the task context or execution outcome, a scenario largely ignored by evaluations targeting explicit threats [101]. Similarly, traditional evaluations often miss "foot-in-the-door" attacks, where harmless initial requests establish a trajectory that makes agents significantly more susceptible to subsequent malicious directives, highlighting the fragility of agents that seldom re-evaluate their actions mid-execution [84]. The reliance on simplistic user tasks in benchmarks like InjecAgent and AgentDojo, which often involve short trajectories and limited tool sets, fails to stress-test the robustness of agents against complex, long-horizon execution flows where errors compound [270]. Consequently, the community faces a critical gap where defenses optimized for these flawed benchmarks fail to provide adequate protection against the sophisticated, multi-stage, and context-aware attacks that characterize the actual deployment of autonomous agents [45].

Defining the Trust Gap and Scope of Review

The transition of Large Language Models (LLMs) from passive conversationalists to autonomous agents capable of executing complex workflows has fundamentally altered the cybersecurity landscape, introducing a critical systemic vulnerability known as the "Trust Gap." This gap arises when agents, primarily evaluated in benign or static settings, fail to maintain necessary skepticism toward tool outputs and external environments, leading to the execution of unauthorized actions. The core of this vulnerability lies in indirect prompt injection (IPI), where malicious instructions embedded in third-party content—such as emails, web pages, or tool metadata—hijack an agent's reasoning process [270]. Unlike direct attacks targeting the user prompt, IPI exploits the agent's inherent design to treat external data as trusted observations, effectively blurring the line between user intent and adversarial manipulation [22]. Recent systematizations of knowledge highlight that this threat surface extends far beyond simple text injection, encompassing tool poisoning, retrieval-augmented generation (RAG) contamination, and cross-agent manipulation, thereby necessitating a paradigm shift from text safety to execution security [142].

A significant contradiction exists in the current literature regarding the efficacy of existing defenses and the robustness of evaluation benchmarks. Several studies suggest that simple, modular defenses operating at the agent-tool interface, such as input and output firewalls, can achieve near-perfect security on established benchmarks like AgentDojo and InjecAgent [36]. However, this apparent success is increasingly viewed as an artifact of benchmark saturation and simplistic task designs rather than genuine robustness. Research indicates that current benchmarks often lack dynamic, open-ended tasks and fail to include helpful third-party instructions, allowing defenses to adopt a strategy of "over-defense" by blindly ignoring external context [270]. When evaluated against more realistic, dynamic environments that require replanning and discrimination between benign and malicious instructions, these same defenses suffer severe utility drops or fail completely, revealing that the security community may be solving a simplified version of the problem [239]. Furthermore, adaptive attack frameworks have demonstrated the ability to bypass nearly all baseline defenses, including those that appeared effective in static evaluations, by exploiting the agent's chain-of-thought and tool-selection mechanisms [38].

The scope of this review extends to the diverse vectors through which the Trust Gap is exploited, moving beyond traditional prompt injection to include sophisticated supply chain and protocol-level attacks. The emergence of the Model Context Protocol (MCP) as a standard for tool integration has introduced new vulnerabilities, specifically tool description poisoning, where malicious metadata manipulates an agent's planning layer without executing malicious code [41]. Studies show that agents are highly susceptible to such attacks, with success rates exceeding 70% even on frontier models, indicating that safety alignment is ineffective against legitimate tools used for unauthorized operations [95]. Additionally, the risk propagates through multi-agent systems via "Agent Cascading Injection," where a compromised agent exploits inter-agent trust to propagate malicious instructions, causing systemic failures that single-agent defenses cannot mitigate [123]. The threat is further compounded by "stored prompt injection," where adversarial content persists in agent memory or filesystems, influencing future executions long after the initial interaction, transforming an ephemeral model threat into a persistent system-level vulnerability [68].

Addressing the Trust Gap requires moving from ad-hoc filtering to systemic, architecture-level mitigations. While prompting-based and alignment-based defenses offer some protection, they often struggle to balance security with utility in complex, long-horizon tasks [4]. Consequently, recent research advocates for "verify-before-commit" protocols and semantic virtualization, which enforce privilege separation and intent grounding before action execution [294]. Other approaches propose lifecycle-integrated security architectures that weave defense layers directly into the agent's execution harness, addressing structural mismatches that leave agents blind to internal state anomalies [149]. Despite these advances, a fundamental tension remains: many defenses rely on the assumption that tool feedback is trustworthy or that the agent can distinguish context, assumptions that are repeatedly invalidated by adaptive adversaries [119]. This review synthesizes these disparate threads to analyze the trajectory from attack formalization to systemic mitigation, highlighting the critical need for benchmarks and defenses that account for dynamic planning, context-dependent reasoning, and the full breadth of the agentic attack surface.

Taxonomy of Threats: From Prompt Injection to Environmental Deception

The threat landscape confronting Large Language Model (LLM) agents has evolved from simple text-based manipulations into a complex ecosystem of supply-chain compromises, protocol-level exploits, and environmental deceptions. At the foundational level, Indirect Prompt Injections (IPI) remain a pervasive vector, where adversaries embed malicious instructions within third-party content such as emails, web pages, or retrieved documents to hijack agent control flow without direct user interaction [22]. These attacks are particularly insidious because they can be query-agnostic, relying on the agent's inherent tendency to treat retrieved data as executable instructions, a vulnerability that persists even in state-of-the-art models [24]. The severity of IPI is amplified in multi-agent systems, where a single compromised node can propagate malicious prompts across the network, behaving analogously to a computer virus in a phenomenon termed "Prompt Infection" [77]. Furthermore, recent research indicates that these injections are becoming increasingly stealthy, with "silent egress" attacks enabling data exfiltration through network requests that leave no trace in the final user-facing response [SILENT EGRESS: WHEN IMPLICIT PROMPT INJECTION MAKES LLM AGENTS LEAK WITHOUT A TRACE].

Beyond direct content injection, the supply chain of agent capabilities presents a critical attack surface through skill and tool poisoning. The introduction of "Agent Skills"-reusable modules containing instructions and code-has created a vector for trivially simple yet devastating injections, where malicious logic hidden in markdown files can bypass system guardrails and execute unauthorized actions like data exfiltration [57]. This vulnerability extends to the broader ecosystem of tool integration, specifically within the Model Context Protocol (MCP). Adversaries can engage in Tool Description Poisoning, injecting malicious directives into the metadata or "manuals" of tools rather than their executable code, thereby manipulating the agent's planning layer to invoke legitimate tools for unauthorized purposes [41]. Systematic evaluations reveal that capable models are often more susceptible to these attacks due to their superior instruction-following adherence, with success rates exceeding 70% in realistic MCP environments [95]. Additionally, function libraries in autonomous systems, such as those used in driving agents, can be poisoned to induce cascading errors in perception and planning, demonstrating that upstream vulnerabilities in shared resources pose existential risks to safety-critical applications [2].

A distinct and emerging category of threats involves adversarial environments that manipulate the agent's perception of reality, effectively constructing a "fake world" to deceive the agent into making erroneous decisions. This form of environmental deception, or Adversarial Environmental Injection, exploits the trust agents place in external tools and search results, leading to epistemic drift where the agent adopts false beliefs or enters infinite loops [11]. In the context of web agents, this manifests through the manipulation of the HTML accessibility tree or the injection of corrupted memory states, allowing attackers to hijack navigation flows or persist malicious intents across sessions [59]. The concept of "Sleepers Attacks" further illustrates the danger of persistence, where adversarial content lies dormant in the agent's memory or session context, only to be triggered by a benign future query, transforming ephemeral threats into long-lived system vulnerabilities [195]. Moreover, mobile and computer-use agents face unique risks from untrusted third-party channels, such as fraudulent advertisements or pop-ups, which can serve as low-barrier entry points for privilege escalation and malware installation [234].

The literature also highlights sophisticated evasion techniques that render traditional filtering mechanisms ineffective. Attacks are increasingly employing temporal, spatial, and semantic evasions, fragmenting payloads across interaction turns or concealing them within complex artifacts to bypass detection [42]. Adaptive attack frameworks like IterInject and AdapTools demonstrate that static defenses are insufficient, as attackers can iteratively refine payloads based on feedback to achieve high success rates even against robust guardrails [112]. Furthermore, the reliance on prompt compression to manage context windows has introduced a novel attack surface where adversarial edits can be transformed into malicious instructions during the compression process, evading pre-processing filters [289]. Despite the proliferation of defense strategies ranging from representation engineering to deception-based traps, evaluations consistently show that no single approach currently offers a comprehensive solution without incurring significant utility costs or failing against adaptive adversaries [141]. The consensus across these studies is that the current security posture of agentic systems is fragile, necessitating a shift from model-centric fixes to architectural reforms that enforce strict trust boundaries and provenance tracking [80].

Indirect Prompt Injection (IPI) and Tool Stream Manipulation

Indirect Prompt Injection (IPI) represents a paradigm shift in the security landscape of Large Language Model (LLM) agents, moving the attack vector from direct user manipulation to the exploitation of untrusted external data sources. Unlike direct injections where an adversary interacts with the model interface, IPI occurs when malicious instructions embedded in websites, emails, documents, or tool outputs are retrieved by an agent and executed as legitimate commands. This vulnerability arises because LLM-integrated applications inherently blur the line between data and instructions, allowing adversaries to remotely exploit systems without direct access to the user interface [Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection]. The severity of this threat is amplified in agentic systems that possess tool-use capabilities, file system access, and web browsing permissions, effectively turning retrieved content into executable code that can trigger unauthorized actions ranging from data exfiltration to arbitrary command execution [How Agentic AI Coding Assistants Become the Attacker's Shell].

A critical evolution in IPI research is the transition from query-specific to query-agnostic attack paradigms. Traditionally, successful injections were thought to require specific user queries to trigger malicious payloads, limiting their generalizability. However, recent advancements demonstrate that attackers can leverage invariant system components, such as system prompts and tool descriptions, to reliably execute malicious actions regardless of the user's specific input [8]. By optimizing malicious tool descriptions to align with an agent's native style and conventions, adversaries can achieve high success rates across diverse tasks, significantly expanding the attack surface for coding agents and general-purpose assistants. This query-agnostic nature implies that defenses relying on detecting mismatches between user intent and tool usage may be insufficient if the tool description itself is compromised, as the malicious instruction becomes a permanent part of the agent's operational context. Furthermore, adaptive frameworks like AdapTools have shown that selecting stealthier attack tools and generating adaptive prompts can bypass state-of-the-art defenses, achieving significant improvements in attack success rates while degrading system utility [20].

The vulnerability extends deeply into the structural components of agent ecosystems, particularly regarding "Agent Skills," protocol standards like the Model Context Protocol (MCP), and memory management. Agent Skills, designed to extend capabilities via markdown files, often treat every line as an instruction, allowing trivially simple injections to bypass system-level guardrails and execute harmful scripts without user confirmation [57]. Similarly, vulnerabilities in MCP allow for "implicit tool poisoning," where malicious metadata in tool descriptions induces agents to invoke legitimate high-privilege tools for malicious purposes, often evading detection mechanisms that focus on explicit code execution [225]. These attacks highlight a supply chain risk where third-party skills and tools, often sourced from unverified marketplaces, can introduce persistent backdoors into the agent's reasoning process [40]. Additionally, hidden comment injections in skill files and the exploitation of prompt compression algorithms have introduced new stealthy attack surfaces, where adversarial edits can manipulate compressed prompts to induce malicious behaviors while bypassing traditional filtering methods [289] [39].

The mechanism of IPI is further complicated by the persistence of injected instructions across sessions and their ability to propagate within multi-agent systems. Analogous to cross-site scripting in web security, "stored prompt injection" allows a single successful injection to influence agent behavior long after the initial interaction, transforming an ephemeral model-level threat into a long-lived system-level vulnerability embedded within agent execution state [68]. In multi-agent environments, this risk is compounded by "Prompt Infection," where malicious prompts self-replicate across interconnected agents, behaving like a computer virus and causing system-wide disruption even when individual agents possess robust single-turn defenses [77]. Moreover, the integration of compression algorithms to manage context length has introduced new attack surfaces, where adversarial edits can manipulate compressed prompts to induce malicious behaviors stealthily, bypassing traditional filtering methods [289]. The risk is also exacerbated by "silent egress," where implicit injections in URL previews or metadata induce agents to exfiltrate sensitive runtime context via outbound requests that appear harmless to the user [SILENT EGRESS: WHEN IMPLICIT PROMPT INJECTION MAKES LLM AGENTS LEAK WITHOUT A TRACE].

Defending against IPI presents a complex challenge, often characterized by an "over-defense dilemma" where robust security measures degrade agent utility and latency. Many existing defenses rely on always-on sanitization or strict filtering, which can block benign instructions and hinder task completion, particularly in dynamic environments where

agents must distinguish between helpful and malicious external guidance [10]. Adaptive attacks continue to bypass many state-of-the-art defenses, with studies showing that even sophisticated frameworks can be circumvented by optimizing attack strategies against specific defense mechanisms, revealing critical vulnerabilities in current instruction-following architectures [38]. Consequently, the security community is increasingly recognizing that prompt injection must be treated as a first-class vulnerability requiring architectural-level mitigations, such as explicit hierarchical memory management and semantic virtualization, rather than ad-hoc filtering or model-centric alignment [109]. Emerging frameworks like AgentVisor enforce semantic privilege separation to intercept tool calls, while AGENTSYS utilizes explicit hierarchical memory management to isolate external data, demonstrating that principled architectural changes can reduce attack success rates to near zero while preserving utility [294] [29]. Other approaches, such as IPIGUARD, model task execution as a traversal over a planned Tool Dependency Graph to decouple action planning from external data interaction, significantly reducing unintended tool invocations [6]. Despite these innovations, the consensus remains that no single approach currently achieves high trustworthiness, high utility, and low latency simultaneously, necessitating a shift toward dynamic, context-aware security policies [239].

Skill Poisoning, Supply Chain, and Protocol Vulnerabilities

The transition of Large Language Model (LLM) agents from conversational interfaces to autonomous executors has introduced a sophisticated supply-chain threat landscape centered on "skill poisoning," a vector structurally distinct from traditional indirect prompt injection (IPI). While conventional IPI typically hides malicious instructions within passive data sources like retrieved web pages or emails, skill poisoning exploits the inherent nature of reusable agent capabilities where the carrier itself—whether a skill file, plugin metadata, or tool description—is semantically licensed to guide behavior [5]. This distinction is critical because it renders lexical screening and surface-level semantic detectors brittle; the boundary between legitimate guidance and malicious hijacking becomes indistinguishable at the text level, allowing malicious directives to compete directly with trusted context for the model's attention [5]. Consequently, attackers can embed persistent malicious logic into artifacts that are invoked across numerous downstream tasks, transforming a one-time injection into a long-term compromise of the agent's goal hierarchy [5]. Empirical evaluations confirm this severity, with benchmarks like SKILL-INJECT revealing attack success rates reaching up to 80% for frontier models, enabling harmful actions ranging from data exfiltration to ransomware-like behavior [40].

The emergence of the Model Context Protocol (MCP) as a de facto standard for connecting agents to external tools has significantly amplified this attack surface by standardizing the exchange of tool metadata and descriptions. Research indicates that MCP introduces specific vulnerabilities such as Tool Description Poisoning (TDP), where attackers inject covert instructions into the descriptive "manuals" agents rely on for planning rather than the executable code itself [41]. Empirical evaluations reveal severe susceptibility in this domain; for instance, leading models like GPT-4o have demonstrated nearly 100% attack success rates in high-risk scenarios involving poisoned tool descriptions, while common prompt-guardrail defenses often prove ineffective or counterproductive, a phenomenon termed the "Firewall Fallacy" [41]. Furthermore, the threat extends beyond explicit poisoning to implicit variants, where frameworks like MCP-ITP demonstrate that attackers can optimize tool metadata to induce agents to invoke legitimate, high-privilege tools for malicious ends without ever calling the poisoned tool directly [225]. Large-scale benchmarking via MCPTox, which evaluated agents against hundreds of real-world MCP servers, confirms that more capable models are often paradoxically more susceptible, as their superior instruction-following abilities make them easier to manipulate via these metadata injections [95].

The supply chain risk is further exacerbated by the persistence and mutability of skill artifacts. This vulnerability is not limited to static files; the SkillHarm benchmark introduces the concept of Self-Mutating Poisoning, where an initially benign skill execution silently mutates its own persistent content, deferring harm until a subsequent reuse, thereby evading single-turn detection mechanisms [252]. Additionally, the "Dependency Steering" attack paradigm illustrates how malicious skills can bias coding agents toward installing attacker-controlled packages during benign tasks, effectively weaponizing the software supply chain without modifying model weights or user prompts [186]. The ease of exploitation is underscored by findings that simple markdown edits in agent skill files can bypass system-level guardrails, particularly when users grant "don't ask again" permissions, allowing malicious scripts to execute without further interaction [57].

Mechanistically, successful skill poisoning operates through "attention hijacking," where the model's internal routing shifts disproportionately from trusted task context to the malicious skill span [5]. This internal signal suggests that defense strategies must move beyond text-only filtering toward monitoring the model's hidden states and attention patterns. However, the architectural design of protocols like MCP often lacks fundamental security primitives, such as capability attestation and origin authentication, which allows malicious servers to claim arbitrary permissions and propagate implicit trust [144]. The dynamic nature of tool exposure in protocols like WebMCP introduces further risks, such as Mid-Session Tool Injection (MSTI), where third-party scripts can alter the visible tool set or frame tool roles during an active session, manipulating the agent's perception of its environment in real-time [172].

Despite the proliferation of defense frameworks, the consensus across multiple studies is that current measures are insufficient against adaptive and structural attacks. Defenses relying on static analysis or simple instruction hierarchies fail to address the nuance of instruction-like carriers, while system-level approaches often suffer from an "over-defense dilemma," degrading utility in benign scenarios [10]. The fragility of existing defenses is evident in evaluations where adaptive attacks bypass nearly all baseline protections, and some mitigations even produce counterproductive side effects [22]. Moreover, dynamic benchmarks like AgentDyn reveal that defenses performing well on static tasks fail

catastrophically in open-ended, dynamic planning scenarios, often due to an inability to distinguish helpful instructions from malicious injections [270]. As agents increasingly rely on external, untrusted artifacts for extended functionality, the security community faces the urgent challenge of developing architectural mitigations that enforce context isolation and verify intent without stifling the autonomous capabilities that define modern agentic systems [45].

Adversarial Environmental Injection and Epistemic Drift

The security paradigm for autonomous agents has shifted fundamentally from defending against direct prompt manipulation to mitigating the compromise of the agent's operational environment itself. This evolution, formalized as Adversarial Environmental Injection (AEI), represents a critical vulnerability where adversaries do not merely inject instructions into a user prompt but construct a "fake world" by poisoning the external tools, data sources, and structural interfaces upon which agents rely for grounding [11]. Unlike traditional attacks that target the model's instruction hierarchy directly, AEI exploits an inherent "Trust Gap" in tool-integrated agents: while systems are rigorously evaluated on their ability to use tools correctly in benign settings, they are rarely tested on their skepticism when those tools lie [11]. This environmental deception induces "epistemic drift," a state where agents form false beliefs based on poisoned retrieval results or enter infinite loops due to structural traps, demonstrating that robustness against breadth attacks (poisoned information) and depth attacks (navigational traps) are often orthogonal capabilities [11]. The literature suggests that current benchmarks often yield a false sense of security because they lack the dynamic, open-ended tasks required to reveal these context-dependent failures, leading to defenses that may suppress contextual inputs essential for task completion [270].

The mechanism of AEI frequently manifests through the manipulation of the web accessibility tree and HTML structures, which serve as the primary perception layer for vision-language and navigation agents. Research demonstrates that adversaries can embed universal adversarial triggers within webpage HTML to hijack agent behavior, causing unintended actions such as credential exfiltration or forced ad clicks [59]. This vulnerability is exacerbated in real-world scenarios where attackers leverage existing infrastructure, such as advertising delivery networks, to inject malicious content without needing direct access to the target website or knowledge of the user's specific intent [151]. Such "black-box" environmental injections achieve high success rates by making malicious content appear contextually relevant, thereby bypassing static defenses that assume a controlled environment [151]. Furthermore, the threat extends to privacy leakage, where environmental injections are specifically crafted to adapt to the webpage context, tricking agents into exposing personally identifiable information (PII) during routine tasks like booking flights [276]. The dynamic nature of modern protocols like the Model Context Protocol (MCP) introduces new surfaces for "Mid-Session Tool Injection," where attackers manipulate the tool registry itself rather than just the tool outputs, rendering many static defenses ineffective [172].

A distinct yet related vector involves the poisoning of the agent's memory and skill ecosystems, leading to persistent epistemic drift across sessions. Attacks such as "Sleeper Attacks" and "Environment-injected Trajectory-based Agent Memory Poisoning" (eTAMP) illustrate how a single contaminated observation can persist in the agent's state, remaining dormant until activated by a benign query in a future session [195] [74]. This persistence transforms prompt injection from an ephemeral threat into a long-lived system-level vulnerability, where agents over-trust self-generated reflections or stored experiences that appear locally correct but induce harmful generalization [14] [138]. The danger is compounded in multi-agent systems or those utilizing skill files, where poisoned artifacts can hijack control flow and communication channels, leading to arbitrary code execution or data exfiltration even if individual agents possess robust direct defenses [50] [40].

Defending against AEI presents a unique challenge known as the "over-defense dilemma," where aggressive sanitization of all untrusted content degrades utility and latency in benign scenarios [10]. Traditional defenses often fail because they treat all external inputs as equally suspicious, ignoring the nuance required for context-dependent tasks [239]. To address this, emerging frameworks like CausalArmor propose detecting "dominance shifts" at privileged decision points, triggering sanitization only when untrusted spans exert disproportionate influence over the agent's actions compared to the user's original intent [10]. Similarly, system-level approaches such as IPIGUARD decouple action planning from external data interaction by modeling execution as a traversal over a Tool Dependency Graph, thereby restricting the agent's ability to be swayed by injected instructions during tool invocation [6]. However, the dynamic nature of modern protocols like the Model Context Protocol (MCP) introduces new surfaces for "Mid-Session Tool Injection," where attackers manipulate the tool registry itself rather than just the tool outputs, rendering many static defenses ineffective [172].

The literature reveals a stark contradiction between agent capability and security; highly capable models often exhibit increased vulnerability to environmental deception because their advanced reasoning allows them to better rationalize

poisoned contexts [24] [74]. Moreover, benchmarks that rely on static tasks or lack helpful third-party instructions often yield a false sense of security, as defenses can simply ignore all external input without impacting task completion in simplified environments [270]. Real-world evaluations indicate that when agents face dynamic, open-ended tasks with mixed benign and malicious environmental cues, existing defenses suffer from severe utility drops or fail to detect sophisticated injections [270] [107]. Consequently, achieving robustness against AEI requires moving beyond prompt-level hardening toward architectural solutions that enforce input provenance, capability isolation, and dynamic trust management across the entire agent-environment interface [SILENT EGRESS: WHEN IMPLICIT PROMPT INJECTION MAKES LLM AGENTS LEAK WITHOUT A TRACE] [80].

Multi-Agent Propagation, Infection, and Cascading Failures

The transition from isolated Large Language Model (LLM) deployments to collaborative multi-agent systems (MAS) has fundamentally altered the security landscape, introducing pathways for vulnerabilities to propagate laterally much like biological infections or computer viruses. Unlike single-agent scenarios where threats are typically confined to the input-output loop of one model, MAS rely on inter-agent communication and shared state, creating a fragile ecosystem where a single compromised entity can manipulate the entire network. This phenomenon, termed "Prompt Infection," describes a scenario where malicious prompts self-replicate across interconnected agents, behaving akin to a computer virus that spreads silently while executing harmful actions such as data theft or system disruption [77]. Research indicates that even a single adversarial agent within a system can bypass simple prompting-based defenses, highlighting the inherent fragility of current inter-agent communication structures [19]. The risk is exacerbated by the "Inter-Agent Trust" boundary, where LLMs that successfully resist direct injection or Retrieval-Augmented Generation (RAG) backdoors may still execute identical malicious payloads when requested by peer agents, leading to a 100% compromise rate in tested models under such exploitation [197].

The mechanism of propagation often involves the subtle manipulation of agent roles and the exploitation of trust inherent in collaborative workflows. In Multimodal Large Language Model (MLLM) societies, a single manipulated agent can act as a "wolf," generating prompts that induce other "sheep" agents to output malicious content, effectively infecting the society through indirect propagation [147]. This cascading failure is not limited to text-based instructions; it extends to the execution of arbitrary malicious code, where adversarial content hijacks control flows to invoke unsafe agents, resulting in complete security breaches including data exfiltration from containerized environments [50]. Furthermore, integrity attacks demonstrate how malicious agents can employ subtle prompt manipulation to bias system operations, acting as "Scapegoaters" or "Free-Riders" to mislead evaluation systems and manipulate collaborative agents without disrupting core functionality, thereby bypassing advanced LLM-based monitors [250]. The concept of "Agent Cascading Injection" (ACI) further formalizes this risk, where a compromised agent exploits inter-agent trust to propagate malicious instructions, causing cascading failures that topology-based evaluations often fail to predict [123]. In extreme simulations, this infectious jailbreak can spread exponentially, compromising nearly all agents in a network of one million within fewer than thirty interaction rounds after a single initial breach [110].

The vulnerability of these systems is deeply rooted in their architectural reliance on shared context and memory. Memory poisoning attacks reveal that a single adversarial write to an agent's persistent memory can exert long-term influence, transforming prompt injection from an ephemeral threat into a system-level vulnerability that persists across sessions [138]. This persistence allows for "Sleeper Attacks," where adversarial content remains dormant in the agent state until activated by a benign user query, making detection significantly more difficult [195]. Such attacks are compounded by "Obsessive Experience Poisoning," where agents distill locally correct but non-transferable adversarial experiences into high-priority rules during self-reflection, biasing future decision-making without triggering immediate safety filters [14]. In software development contexts, compromised agents in coding and testing phases pose severe risks, with attacks achieving high success rates by manipulating agents to generate software with concealed malicious capabilities [159]. Similarly, in autonomous driving systems, poisoning the shared function library can manipulate one agent's decisions, triggering cascading errors that mislead other agents in the network [2].

Defensive strategies against these propagation vectors face significant challenges, particularly regarding the trade-off between security and utility. Traditional prompting-based defenses often fail in dynamic, open-ended tasks, suffering from over-defense that degrades system functionality or under-defense that leaves systems vulnerable to sophisticated injections [270]. While some multi-agent defense pipelines claim to achieve 100% mitigation by employing specialized agents in coordinated hierarchies, these solutions are often tested against limited attack categories and may not generalize to real-world adaptive threats [133]. More robust approaches propose system-level architectures, such as "Cognitive Executive Separation" or semantic virtualization, which structurally prevent the reasoning system from executing actions without independent validation, thereby containing the spread of infection even if the reasoning core is compromised [299; 294]. Additionally, frameworks utilizing "Sentinel Agents" for distributed monitoring and anomaly detection offer a promising direction for containing threats within the MAS ecosystem, though their efficacy against coordinated multi-agent attacks requires further validation [281]. Ultimately, the literature suggests that

securing multi-agent systems requires moving beyond model-level guardrails to adopt architectural constraints that limit the blast radius of any single compromised agent.

Attack Vectors Targeting Tool Selection, Resource Integrity, and Memory

The security landscape of Large Language Model (LLM) agents has evolved from simple prompt injection to sophisticated assaults on the fundamental mechanisms of tool selection, resource allocation, and persistent memory. A primary vector involves the manipulation of tool metadata to deceive agents into selecting malicious or inappropriate tools. Adversaries can iteratively optimize tool names and descriptions to bias selection policies, a technique demonstrated by [67], which successfully increased selection rates for specific tools from a baseline of 20% to as high as 81%. Similarly, [46] proposes a black-box framework that generates syntactically valid but highly attractive metadata, achieving attack success rates between 81% and 95% across various scenarios. These metadata poisoning strategies are particularly effective within standardized ecosystems like the Model Context Protocol (MCP). [41] highlights that leading models such as GPT-4o exhibit nearly 100% success rates in high-risk scenarios where malicious instructions are embedded in tool descriptions rather than code. This vulnerability is further systematized in [95], which reveals that more capable models are often more susceptible due to their superior instruction-following abilities, with refusal rates remaining below 3% even for safety-aligned models. Beyond simple selection bias, attackers have developed strategies to saturate the retrieval layer itself. [183] introduces an attack that injects attacker-controlled tools to dominate the top-k retrieval results, effectively hiding benign tools from the agent's context and creating a denial-of-visibility at the ecosystem level.

Beyond simple selection bias, attackers have developed "implicit toxicity" strategies that exploit authorized tools to hijack computational resources or perform unauthorized operations without triggering explicit privilege violations. [15] identifies a class of attacks where malicious tools embed backdoors that activate upon specific triggers, effectively parasitizing the user's compute budget for unauthorized workloads while maintaining normal functionality for the user. This concept of implicit trust is further explored in [225], which automates the generation of poisoned tools that induce agents to invoke legitimate, high-privilege tools for malicious ends. This approach achieves high success rates while keeping detection rates extremely low, demonstrating that security mechanisms focusing solely on explicit privilege escalation fail to detect authorized but harmful behaviors. Furthermore, [75] illustrates how chaining together seemingly harmless tool calls can collectively enable harmful operations, a threat that isolated prompt defenses often miss. The vulnerability of the tool interface itself is also evident in runtime manipulation, where [172] describes Mid-Session Tool Injection attacks that alter the tool set visible to the agent during an active session, causing malicious tools to appear legitimate.

The persistence of agent memory introduces a critical, long-term attack surface where a single successful injection can compromise future sessions. [52] demonstrates that injecting a small number of crafted records disguised as technical facts can reshape an agent's contextual perception, leading it to autonomously select attacker-preferred tools with success rates up to 85.9%. This threat extends to environment-injected poisoning, where [74] shows that viewing a manipulated product page can silently poison an agent's memory, activating during future tasks on different websites. The concept of "sleepers" attacks is formalized in [285], where poisoned memories remain dormant until retrieved in a specific context, causing attacker-intended actions in up to 89% of evaluations. Additionally, [180] identifies that memory retrieval can dominate control flows, forcing unintended tool usage even against explicit user instructions. The vulnerability of self-evolving agents is highlighted in [229], where indirect exposure to poisoned web content converts a one-time injection into a persistent compromise. Even the process of reflection and self-improvement can be weaponized, as shown in [14], where adversaries induce agents to generate locally correct but harmful generalizations that bias future rule formation. The fragility of current architectures against these persistent threats is underscored by [29], which notes that conventional agents indiscriminately accumulate all tool outputs in memory, allowing injected instructions to persist throughout the workflow. Ultimately, securing agentic systems requires addressing not only the model's reasoning but also the integrity of tool metadata, the isolation of computational resources, and the provenance of persistent memory states.

Adversarial Tool Selection and Metadata Poisoning

The tool selection pipeline in Large Language Model (LLM) agents, typically comprising a retrieval stage followed by a selection stage, represents a critical and increasingly exploited attack surface. Adversaries can manipulate this process not by altering the agent's core model weights, but by injecting deceptive tools with optimized metadata, saturating retrieval results, or poisoning the agent's memory, leading to unauthorized data access, denial of service, or complete workflow hijacking. A primary vector involves the manipulation of tool names, descriptions, and schemas to bias the agent's selection mechanism. Research demonstrates that by iteratively optimizing tool metadata, attackers can significantly increase the selection rate of malicious tools. For instance, [67] introduces a lightweight automatic attack that elevates tool selection rates from a baseline of approximately 20% to as high as 81%, revealing how implicit competition in tool ecosystems can be weaponized to create distributional shifts in tool usage. Similarly, [46] proposes a black-box framework that generates syntactically valid yet highly attractive metadata, achieving attack success rates between 81% and 95% across various scenarios without requiring prompt injection or access to model internals. These findings underscore that metadata manipulation constitutes a potent, stealthy threat surface that often bypasses standard prompt-level defenses. This approach is further formalized in [127], which presents ToolHijacker, a method that formulates the crafting of malicious tool documents as an optimization problem, significantly outperforming manual injection techniques in no-box scenarios.

Beyond individual tool optimization, adversaries can target the retrieval layer itself to dominate the candidate set presented to the agent. [183] identifies a failure mode termed "Top-k domination," where an attacker injects a swarm of Sybil tools strategically positioned in the embedding space to semantically cover a wide range of user queries. This approach overwhelms the retrieval mechanism, pushing all benign tools out of the top-k results and forcing the agent to reason exclusively over an attacker-curated universe. Empirical evaluations show that ToolFlood can achieve up to 95% domination with an injection rate of only 1%, effectively rendering selection-time safeguards useless since benign tools are never retrieved. This fragility is further quantified by [23], which introduces TOOLCERT, a statistical framework for certifying tool selection robustness. Their analysis reveals that under adaptive attacks involving misleading metadata or retrieval saturation, the certified lower bound on accuracy for state-of-the-art agents drops close to zero, representing a performance decline of over 60% compared to benign settings. The theoretical implications of this saturation suggest that embedding-based retrieval, while efficient, lacks inherent robustness against geometrically optimized adversarial inputs.

The threat landscape extends to the protocols standardizing agent-tool interactions, such as the Model Context Protocol (MCP). [95] highlights widespread vulnerabilities in real-world MCP servers, where malicious instructions embedded in tool metadata lead to attack success rates exceeding 70% for prominent models. The study notes a counterintuitive trend where more capable models are often more susceptible due to their superior instruction-following abilities. Building on this, [225] demonstrates that implicit tool poisoning—where the poisoned tool remains uninvoked but induces the agent to misuse legitimate high-privilege tools—can achieve success rates up to 84.2% while evading detection. Furthermore, [41] exposes severe vulnerabilities in leading models like GPT-4o, which exhibited nearly 100% attack success rates in high-risk scenarios, and identifies a "Firewall Fallacy" where common prompt-guardrail defenses prove ineffective or even counterproductive. These studies collectively indicate that standardization efforts, while improving interoperability, have inadvertently created uniform attack surfaces that are easily exploited across diverse agent implementations.

Attackers also exploit the agent's long-term memory and dynamic runtime environments to sustain malicious influence. [52] presents the first attack biasing tool selection by poisoning the agent's long-term memory with crafted records disguised as operational policies, achieving up to 85.9% success with only three injected records. In dynamic web environments, [172] identifies Mid-Session Tool Injection (MSTI), where third-party scripts alter the tool set or definitions visible to the agent during an active session, successfully hijacking tool selection or framing malicious tools as legitimate steps. Additionally, the supply chain risk is exacerbated by tools containing malicious code or steering dependencies, as shown in [136] and [186], which demonstrate that existing malware detection and skill scanners fail to identify these sophisticated threats. The cumulative evidence from these studies indicates that the tool selection pipeline is fragile against adversarial metadata, retrieval saturation, and memory poisoning, necessitating a shift from prompt-level filtering to structural, execution-layer defenses that verify tool integrity and provenance. The insufficiency

of current defenses is further corroborated by [127], which shows that existing prevention and detection mechanisms fail to mitigate optimized tool document injections, highlighting the urgent need for new defense strategies that operate beyond the textual context window.

Implicit Toxicity, Resource Hijacking, and Denial of Efficiency

The emergence of the Model Context Protocol (MCP) and similar frameworks has standardized the integration of external tools into Large Language Model (LLM) agent ecosystems, fostering a vibrant marketplace of community-built functionalities. However, this openness introduces a critical security assumption: implicit trust in third-party tool providers. This trust boundary is exploited by a newly identified class of attacks termed "implicit toxicity," where malicious behaviors occur entirely within the allowed privilege scope, bypassing traditional security mechanisms that focus on explicit violations [15]. Unlike prior attacks that seek to compromise the host or generate harmful outputs through privilege escalation, implicit toxicity leverages legitimate tool interfaces to covertly divert computational resources or manipulate agent reasoning while maintaining normal functionality for the user [15]. This paradigm shift reveals a systemic blind spot where security mechanisms fail to detect authorized but harmful behaviors, highlighting the urgent need for computational provenance and resource attestation [15] [256].

A primary manifestation of this threat is resource hijacking, exemplified by the "LeechHijack" attack. In this scenario, an adversarial MCP tool embeds a benign-looking backdoor that activates upon predefined triggers to establish a command-and-control channel. Through this channel, attackers inject additional tasks that the agent executes as part of its normal workflow, effectively parasitizing the user's compute budget for unauthorized workloads [15]. Experimental implementations across major LLM families demonstrate that LeechHijack achieves a high success rate with minimal resource overhead, rendering it practically undetectable by existing code auditing or runtime monitoring frameworks [15]. Parallel to resource expropriation is the threat of denial-of-efficiency, where attacks force agents into convoluted reasoning paths to exhaust computational budgets. The "Sponge Tool Attack" (STA) disrupts agentic reasoning by rewriting input prompts to convert concise trajectories into unnecessarily verbose ones, resulting in substantial computational overhead while preserving the original task semantics [243]. Similarly, "LoopTrap" exploits the iterative execution loops of modern agents by injecting malicious prompts that distort termination judgments, causing agents to believe tasks remain incomplete and leading to unbounded computation [61]. These attacks do not necessarily alter the final output but degrade system efficiency and availability, representing a subtle yet potent form of denial-of-service [243] [61].

The mechanism of implicit toxicity often relies on the manipulation of tool metadata and descriptions rather than executable code. Research indicates that agents exhibit a blind reliance on tool descriptions, making them susceptible to "Tool Description Poisoning" (TDP) where malicious instructions are covertly injected into the descriptive metadata—the "manual" an agent relies on for planning [41]. Automated frameworks like MCP-ITP have demonstrated the ability to generate poisoned tool metadata that induces agents to invoke legitimate but high-privilege tools for malicious operations, achieving high attack success rates while evading detection [225]. Furthermore, attacks such as "ToolTweak" and "Attractive Metadata Attack" show that adversaries can systematically bias tool selection by iteratively manipulating tool names and descriptions, gaining unfair advantage over equally capable alternatives and shifting the distribution of tool usage [67] [46]. This vulnerability is exacerbated by the fact that more capable models are often more susceptible, as the attack exploits their superior instruction-following abilities [95].

The vulnerability extends to the dynamic nature of tool surfaces in web-based environments. "Mid-Session Tool Injection" (MSTI) attacks leverage third-party scripts to inject malicious tools or alter tool definitions during an active session, disrupting intended functionality while the task appears to complete normally [172]. This dynamic exposure creates a new attack surface where the tool set itself becomes a target for manipulation, challenging the assumption that available tools within a session are static and trusted [172]. Additionally, "Implicit Malicious Behavior Injection Attacks" (IMBIA) in multi-agent systems demonstrate how compromised agents can generate software with concealed malicious capabilities beneath seemingly benign applications, further complicating the detection of implicit threats [159]. These attacks can persist across interactions, formalized as "Sleeper Attacks," where adversarial content remains dormant in the agent state until activated by a benign user query [195].

Defending against these implicit threats proves challenging because they operate within the bounds of authorized behavior. Traditional prompt-level defenses and safety alignments often fail when user instructions are benign, as safety mechanisms primarily activate in the initial steps and rarely re-engage during subsequent execution [101]. Moreover, the "Firewall Fallacy" suggests that common prompt-guardrail defenses can be counterproductive, failing to prevent malicious actions that utilize legitimate tools for unauthorized operations [41]. Consequently, there is a growing

consensus that robust defense requires moving beyond input sanitization toward architectural solutions such as computational provenance, resource attestation, and capability isolation [15] [256]. The systemic nature of these vulnerabilities underscores the necessity for a paradigm shift in securing agentic ecosystems, focusing on the integrity of the tool selection process and the verification of resource utilization rather than solely on output filtering [41] [256].

Conclusion and Future Directions

The comprehensive review of agentic security literature confirms a definitive paradigm shift: protecting autonomous systems requires moving beyond probabilistic, model-centric safety measures to deterministic, system-level governance. While early defenses focused on filtering harmful text, the evolution of agents into executive entities capable of tool use, memory persistence, and multi-agent collaboration has rendered such approaches insufficient against execution-layer threats like indirect prompt injection, tool description poisoning, and cross-session memory corruption [7]. The consensus indicates that structural vulnerabilities, where semantically benign inputs trigger irreversible actions, cannot be resolved by alignment alone but demand architectural reforms such as cognitive-executive separation and zero-trust authorization [299] [1].

A critical challenge identified is the "over-defense dilemma," where rigid security constraints degrade agent utility and latency. Emerging solutions address this by employing causal attribution and selective intervention, triggering defenses only when untrusted data exerts disproportionate influence over decision-making, thus balancing robustness with operational fluidity [10] [31]. Furthermore, the inadequacy of static benchmarks has driven the development of dynamic, open-ended evaluation frameworks that better capture multi-turn interactions, adaptive adversaries, and the nuances of human oversight, emphasizing the need for standardized autonomy metrics over binary attack success rates [270] [264].

Looking ahead, the security of agentic ecosystems depends on holistic lifecycle governance that integrates provenance tracking, immutable audit logging, and formal verification to mitigate risks in multi-agent, environmental, and physical domains [284]. As agents become increasingly embodied and interconnected, future research must prioritize defenses against lateral propagation, epistemic drift, and policy-executable jailbreaks, ensuring that safety is treated as an architectural invariant rather than an emergent property of model training [201] [203].